1 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`) that *produces* the satellite-side corrections the single-receiver user modes consume. Where every other mode in this reference *differences away* the satellite clock and hardware biases (the differencing section) or imports them as known SSR quantities (the SSR-corrections section), the network producer turns that relationship around: it estimates the satellite clock and the satellite *phase* and *code* biases *explicitly*, jointly across a network of stations whose coordinates are known, and emits them in IGS-SSR convention for a downstream consumer. The closed loop is deliberate — the same single-receiver PPP-RTK engine documented elsewhere becomes the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core — the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`) — is implemented and validated on a synthetic noise-free network (truth recovery up to the declared S-transform). Mode wiring is live: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`src/core/measurement/MeasurementBuilder.cpp`), the explicit satellite phase-bias state in `filter/StateManager`, and the dispatch in `Application::run` all exist. The live epoch loop `Application::runNetwork` — per-station design-row assembly from each station's `MeasurementModel`, the between-epoch time update, and the product emitter — is a scaffold at the time of writing: it logs a "not yet implemented" error and returns non-zero (`src/core/app/ApplicationNetwork.cpp`). The mathematics below describes the implemented estimator core and the locked design; where a piece is staged rather than wired, it is flagged as such.

1.1 Joint multi-station observation model

The producer runs on the same *undifferenced, uncombined* observations as single-receiver PPP (the observation-equations section), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The defining structural fact is the partition of the unknowns into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias β^i_s. These columns are *identical* across every station — a satellite's clock and hardware delays do not depend on who is observing it. (Orbits are held fixed to a precise product here; orbit determination is out of scope for the producer.)
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t_r^{(g)}$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier. Station coordinates are *known* (the network constrains them), so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (the observation-equations section) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{1}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances.

Crucially there is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. That sparsity is the structural engine of Section 1.4.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode::None` (`MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle that may be present in a network config. Differencing would cancel exactly the satellite-side terms the producer is trying to *estimate*; the network instead carries them as explicit columns and resolves the resulting rank deficiency with a minimum-constraint datum (Section 1.3).

1.2 The explicit satellite phase-bias state

The single defining contrast with the user side is the treatment of the satellite phase bias b_s^i (here written β_s^i once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (the differencing section) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer is what *generates* that known product, so it must carry the satellite phase bias as a genuine estimated state. The state vector therefore gains a per-(satellite, signal) phase bias column,

$$\beta_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{2}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept *deliberately disjoint* from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two namespaces from colliding (`include/filter/StateKeySchema.hpp`). Its process noise is a tiny random walk (`process_noise.sat_phase_bias`, default two orders of magnitude below the code-bias density), reflecting that satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column deaf to genuine slow drift (`src/core/filter/StateManager.cpp`).

Remark 1. The estimated β_s^i is not the "raw" satellite phase hardware delay — that quantity is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [1]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. This lumping is exactly why a *consistent* phase-bias product matters (the SSR-corrections section): the producer's datum and the consumer's integer recovery must be the same S-system.

1.3 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (1) is rank deficient. The same linear dependencies that the user side disposes of by differencing are present here as exact *gauge freedoms* of the joint normal matrix: there is no observation that distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks, nor the split of an absolute phase bias between the satellite column and the per-station ambiguity datum, nor an inter-system bias (ISB) common to a constellation. These are precisely the S-system ambiguities of [2]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The chosen resolution is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a hard structural elimination of columns.

Concretely, each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution is moved to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c = \text{value}$ holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam), leaving the estimator's own reduced system pristine so per-station back-substitution (Section 1.4) stays valid. The pinned diagonal is set commensurate with the reduced normals' own scale rather than to unity, so that a genuinely *under*-specified datum still surfaces as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, if $\boldsymbol{x}$ collects the rank-deficient undifferenced parameters and $\boldsymbol{S}$ is the S-basis selection, the emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$; choosing the datum *is* choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges** — the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum — are the elements that touch the shared satellite columns. These go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** — a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9) — live on that one station's private block. They are realised by the *caller* as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors are also what make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 1.4).

Remark 2. A rigorous null-space rank gate — SVD the pre-constraint reduced normals and assert that the count of small singular values equals the *declared* S-basis dimension — is the planned W5 slice (`NetworkSbasis`) and is not yet present. The `Datum` in the current estimator is the minimal pin set the solve needs; the positive-definiteness guard catches the exact-deficiency (under-specified datum) case but does not yet certify the rank against a declared dimension. This is recorded as an open item rather than claimed as done.

1.4 Per-station Schur reduction (arrowhead normals)

Stacking Equation (1) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system, because the station–station off-diagonal blocks are exactly zero (stations couple only through $\boldsymbol{x}_s$):

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{3}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \tag{4}$$

and $\boldsymbol{u}_s^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$, $\boldsymbol{u}_r^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead structure is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added; the elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}} \right) \; + \; \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{5}$$

$$\tilde{\boldsymbol{u}}_{s} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{u}_{s}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{u}_{r}^{r} \right) \; + \; \boldsymbol{u}_{s}^{\mathrm{prior}}, \tag{6}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation, not an approximation: solving Equations (5) and (6) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = \left(\boldsymbol{N}_{rr}^{r}\right)^{-1}\left(\boldsymbol{u}_{r}^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\,\boldsymbol{x}_s\right), \tag{7}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost scales linearly in the station count and the per-station eliminations parallelise (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (5) — soft physical priors and any additive exact-gauge contributions — are added pre-reduction via `addSharedInfo` and carry through the Schur complement unchanged.

Remark 3. This arrowhead-with-exact-reduction topology mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver, specialised so that the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. The design panel rejected a two-stage covariance-form alternative precisely because the normal-equation Schur form is exact and cheaper here.

1.5 Constellation scope

The producer estimates GPS and Galileo, with BeiDou supported on the same footing; **GLONASS is excluded**. The reason is structural, not a tuning choice: GLONASS is FDMA, so each satellite transmits on a slightly different frequency and the receiver inter-channel (hardware) bias does *not* cancel cleanly across satellites the way it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 1.2 and 1.3) assumes a per-signal bias common across a constellation's satellites; the per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the datum. GLONASS is therefore omitted from the network solve by design (`docs/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. (This is distinct from the float-only GLONASS support the user engine carries for positioning; the network *producer* simply does not generate GLONASS bias products.)

1.6 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is exactly the SSR stream the single-receiver engine already knows how to consume: the satellite clock refines $-c\,\mathrm{d}t^{i}$ and the observable-specific biases refine $-B_s^i$ and $-b_s^i$ as *known* corrections, restoring the integer ambiguity property exactly as described in the SSR-corrections section.

The validation strategy follows directly from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. Holding one station *out* of the network solve and then positioning it as a user against the produced corrections turns the consumer into a closed-loop product test: the held-out station's recovered position and fixed ambiguities certify that the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer flows through the standard PPP path (it is not a separate estimator); only the producer takes the joint multi-station pipeline of this section.

References

[1] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.